Live Disruption & Safety Alerts — Hackathon Problem Statement

1. The problem

When severe weather or a disaster threatens a place, people turn to search: "weather alerts in Houston", "flood warning in Colorado", "is there a tornado warning near me". Official warnings exist, but they're scattered across agencies, written as long technical bulletins, and split by zones and offices. They rarely answer the simple question people are asking:

Is anything happening there right now, how serious is it, where exactly, and until when?

2. The challenge

Build a standalone hyperDart component that appears when someone searches for live alerts or disruptions in a place, and gives them a clear, trustworthy answer in seconds.

At minimum, your component should handle:

- **Active alerts for a place:** what's happening, how serious it is, where exactly, and until when
- **Specific hazards:** queries for one kind of alert, such as floods, heat or tornadoes
- **"Near me" queries**, using the searcher's location
- **No alerts, and places outside coverage:** an honest answer that never gives false reassurance

Apart from the basic requirements, you can add more features, like:

- **A map** of where each alert applies
- **Plain-language guidance** on what to do now, based on the official instructions
- **A timeline or countdown** of when alerts start and end, in the place's local time
- **Other official sources** for places outside the US

> **What you show, how you show it, and how users interact with it are up to you. We're looking for original ideas, a unique design and solid engineering, not a copy of an official bulletin page. The best components answer the question at a glance and then let people dig deeper.**

3. What we provide

Trigger keywords and regex

Add these to `hyperdart.config.js`. You may extend them. Places come from the `HD_LOCATION` collection type (cities, states and countries). To trigger on other kinds of input too, add regexes using any collection from the Entities docs.

```
keywords: ['weather alerts', 'weather alert', 'weather warnings', 'weather
warning', 'severe weather', 'tornado warning', 'flood warning', 'heat
advisory', 'winter storm warning', 'safety alerts']
```

```
regex: [
  '(weather|severe\\s+weather|safety|emergency)\\s+(alerts?|warnings?
|advisories)\\s+(in|for|near|at)\\s+HD_LOCATION(__\\w+)?.*',

'(tornado|flood|flash\\s+flood|heat|winter\\s+storm|storm|wind|fire\\s+weathe
r|hurricane|air\\s+quality)\\s+(alerts?|warnings?|watch(es)?
|advisor(y|ies))\\s+(in|for|near|at)\\s+HD_LOCATION(__\\w+)?.*',
  'is\\s+there\\s+(a|an|any)\\s+.*(warning|alert|watch|advisory)\\s+
(in|for|near)\\s+HD_LOCATION(__\\w+)?.*',
  'HD_LOCATION(__\\w+)?\\s+(weather\\s+)?(alerts?|warnings?|advisories).*',
  '(weather\\s+)?(alerts?|warnings?)\\s+(near|around)\\s+me.*'
]
```

searchData

Your component receives a `searchData` prop with the place found in the query, including its coordinates, and the searcher's approximate position in `userLocation` (for "near me" queries). Depending on the template it may arrive as an array or a single object. Example for `weather alerts in Houston` (trimmed):

```
[{
  "query": "weather alerts in Houston",
  "entities": [{
    "word": "Houston",
    "collectionName": "CITIES",
    "collectionType": "HD_LOCATION",
    "entityInfo": {
      "geo": {
        "lat": 29.76328, "long": -95.36327,
        "city": "Houston", "state": "Texas", "stateAbb": "TX",
        "country": "United States", "countryCode": "US", "tz":
"America/Chicago"
      }
    }
  }],
  "userLocation": {
    "position": { "coords": { "latitude": 26.4631, "longitude": 80.3479 },
"countryCode": "IN" },
    "source": "ip"
  }
}]
```

Data source

The US National Weather Service (NWS) Alerts API publishes all active official warnings, watches and advisories for the United States and its territories. It's free, needs no API key, and accepts calls from the browser. You may use other official sources too, including ones for other countries.

```
GET https://api.weather.gov/alerts/active?point={latitude},
{longitude}&status=actual&message_type=alert,update
```

Send the header `Accept: application/geo+json`. Instead of `point`, you can filter by `area` (a state code such as `TX`) or `zone`, and narrow results with `severity`, `urgency`, `certainty` or `event`. Each alert includes its event, severity, urgency, affected areas, start and end times, headline, description and recommended actions.

This is a free public service: test your calls early, and expect quiet days with no alerts, limits and failures.

4. Example queries

User query	Component shows
`weather alerts in Houston`	Active alerts for Houston, or a clear "no active alerts"
`flood warnings in Colorado`	Flood warnings across Colorado
`severe weather alerts in Alaska`	Severe alerts across Alaska
`is there a heat advisory in Phoenix`	A direct yes or no, with details if yes
`weather alerts near me`	Alerts for the searcher's location
`weather alerts in Delhi`	An honest answer for a place the data source doesn't cover

5. Submission

1. Source code in a public GitHub repository. Include a `.gitignore` so `node_modules` and `dist` aren't committed (the templates don't ship one).
2. The component published to the hyperDart dev registry.
3. A README covering your idea, design decisions, data sources, libraries, and how you built it (including any AI tools you used and what you wrote or changed yourself).
4. A 2–3 minute demo video on YouTube.

Add both links to `hyperdart.config.js`, using the field names already in your template:

```
github: 'https://github.com/<user>/<repo>',     // repo root, not a
/tree/main link
demovideo: 'https://www.youtube.com/watch?v=<id>'  // or
https://youtu.be/<id>
```

Publish with plain `npm publish`. It runs the prepublish validation (`scripts/validate-publish.js`), which checks keywords, regex, build output and both links. **Submissions published with validation skipped (for example `npm publish --ignore-scripts`, or by removing the `prepublishOnly` script) won't be evaluated.**

Before your first publish, create the package in the Partner Portal (My packages → Create package) and set your token with the command given in portal.

6. Judging criteria

Criterion	Weight	What judges look for
Design and creativity	25%	An original, unique design and interaction that works in sidebar and mainline, light and dark
Usefulness and clarity	20%	How quickly and clearly the component answers "is anything happening, how serious, where, until when"
Accuracy and responsibility	15%	Faithful use of official data; no false reassurance
Robustness	15%	Behaviour across different places and queries, quiet days, places outside coverage and API failures
Understanding (live review)	15%	Explaining the code and making a small change live, for example adding a query pattern
Code quality and README	10%	Readable code and a clear README

Extra features can lift a score, but only once the basic requirements work.

7. Resources

Resource	Link
hyperDart developer docs	hyperdart.com/docs/docs/intro
Templates (`npm create hyper-dart@latest`)	Templates
searchData and props	Props received
Keywords, regex and config	hyperdart.config.js
Publishing	Publish
Entities (collection names and types for regexes)	Entities
NWS API documentation	weather.gov web API
NWS OpenAPI specification	api.weather.gov/openapi.json